1.

C++ code

```
#include<iostream>
#include<cmath>
using namespace std;
#define pi acos(-1.0)
class Point {
private:
     double X, Y;
public:
     Point(double xx = 0, double yy = 0) : X(xx), Y(yy) { cout << "構造了一個Point類對象" << endl; }
     Point(const Point& P) :X(P.X), Y(P.Y) { cout << "複制構造了一個Point類對象" << endl; }
     virtual double GetArea() { return 0.0; };
};
class Circle : public Point {
private:
     double R;
public:
     Circle(double xx = 0, double yy = 0, double rr = 0) : Point(xx, yy), R(rr) { cout << "構造了一個Circle類對象" << endl; }
     Circle(const Point& P, double rr = 0) :Point(P), R(rr) { cout << "構造了一個Circle類對象" << endl; }
     Circle(const Circle& C) :Point(C), R(C.R) { cout << "複制構造了一個Circle類對象" << endl; }
     double GetArea() { return pi * R * R; }
};
class Rectangle: public Point {
private:
     double width, height;
public:
     Rectangle(double xx = 0, double yy = 0, double ww = 0, double hh = 0) :Point(xx, yy), width(ww), height(hh) { cout << "構造了一個Rectangle類對象" << endl; }
     Rectangle(const Point& P, double ww = 0, double hh = 0) :Point(P), width(ww), height(hh) { cout << "構造了一個Rectangle類對象" << endl; }
     Rectangle(const Rectangle& Rec) :Point(Rec), width(Rec.width), height(Rec.height) { cout << "複制構造了一個Rectangle類對象" << endl; }
     double GetArea() { return width * height; }
};
class Square :public Rectangle{
public:
     Square(double xx = 0, double yy = 0, double length = 0) :Rectangle(xx, yy, length, length) { cout << "構造了一個Square對象" << endl; }
     Square(const Point& P, double length = 0) :Rectangle(P, length, length){ cout << "構造了一個Square對象" << endl; }
     Square(const Square& Squ) : Rectangle(Squ){ cout << "複制構造了一個Square對象" << endl; }
     double GetArea() { return Rectangle::GetArea(); }
};
void f(Point& p) {cout << "f::面積是" << p.GetArea() << endl;}
int main() {
     Square Squ1(0, 0, 1), Squ2(Point(2,2), 3);
     Rectangle Rec1(2, 3, 2, 4), Rec2(Point(5, 6), 4, 5);
     Circle C1(5, 6, 1), C2(Point(1, 2), 2);
     Point P1(1, 2), * P[4] = {&Point(P1), &Circle(C2), &Rectangle(Rec2), &Square(Squ2)};
     f(P1); f(C1); f(Rec1); f(Squ1);
     for (int i = 0; i < 4; i++) {
          cout << "P[" << i << "]->" << "面積是" << P[i]->GetArea() << endl;
     }
}
```

Result

```
構造了一個 Point 類對象
構造了一個 Rectangle 類對象
構造了一個 Square 對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
構造了一個 Rectangle 類對象
構造了一個 Square 對象
構造了一個 Point 類對象
構造了一個 Rectangle 類對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
構造了一個 Rectangle 類對象
構造了一個 Point 類對象
構造了一個 Circle 類對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
構造了一個 Circle 類對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
複制構造了一個 Point 類對象
複制構造了一個 Circle 類對象
複制構造了一個 Point 類對象
複制構造了一個 Rectangle 類對象
複制構造了一個 Point 類對象
複制構造了一個 Rectangle 類對象
複制構造了一個 Square 對象
f::面積是 0
f::面積是 3.14159
f::面積是 8
f::面積是 1
P[0]->面積是 0
P[1]->面積是 12.5664
P[2]->面積是 20
P[3]->面積是 9
```

2.

C++ code

```
#include<iostream>
#include<cmath>
using namespace std;
#define pi acos(-1.0)

class Point {
private:
    double X, Y;
public:
    Point(double xx = 0, double yy = 0) : X(xx), Y(yy) { cout << "構造了一個Point類對象" << endl; }
    Point(const Point& P) :X(P.X), Y(P.Y) { cout << "複制構造了一個Point類對象" << endl; }
    virtual ~Point() { cout << "析構了一個Point類對象" << endl; }
    virtual double GetArea() { return 0.0; };
};

class Circle : public Point {
private:
    double R;
public:
    Circle(double xx = 0, double yy = 0, double rr = 0) : Point(xx, yy), R(rr) { cout << "構造了一個Circle類對象" << endl; }
    Circle(const Point& P, double rr = 0) :Point(P), R(rr) { cout << "構造了一個Circle類對象" << endl; }
    Circle(const Circle& C) :Point(C), R(C.R) { cout << "複制構造了一個Circle類對象" << endl; }
    ~Circle() { cout << "析構了一個Circle類對象" << endl; }
    double GetArea() { return pi * R * R; }
};

class Rectangle : public Point {
private:
    double width, height;
public:
    Rectangle(double xx = 0, double yy = 0, double ww = 0, double hh = 0) :Point(xx, yy), width(ww), height(hh) { cout << "構造了一個Rectangle類對象" << endl; }
    Rectangle(const Point& P, double ww = 0, double hh = 0) :Point(P), width(ww), height(hh) { cout << "構造了一個Rectangle類對象" << endl; }
    Rectangle(const Rectangle& Rec) :Point(Rec), width(Rec.width), height(Rec.height) { cout << "複制構造了一個Rectangle類對象" << endl; }
    ~Rectangle() { cout << "析構了一個Rectangle類對象" << endl; }
    double GetArea() { return width * height; }
};

class Square :public Rectangle {
public:
    Square(double xx = 0, double yy = 0, double length = 0) :Rectangle(xx, yy, length, length) { cout << "構造了一個Square對象" << endl; }
    Square(const Point& P, double length = 0) :Rectangle(P, length, length) { cout << "構造了一個Square對象" << endl; }
    Square(const Square& Squ) : Rectangle(Squ) { cout << "複制構造了一個Square對象" << endl; }
    ~Square() { cout << "析構了一個Square對象" << endl; }
    double GetArea() { return Rectangle::GetArea(); }
};

void f(Point& p) {
    cout << "f::面積是" << p.GetArea() << endl;
}
```

Result

```
構造了一個 Point 類對象
構造了一個 Rectangle 類對象
構造了一個 Square 對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
構造了一個 Rectangle 類對象
構造了一個 Square 對象
析構了一個 Point 類對象
構造了一個 Point 類對象
構造了一個 Rectangle 類對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
構造了一個 Rectangle 類對象
析構了一個 Point 類對象
構造了一個 Point 類對象
構造了一個 Circle 類對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
構造了一個 Circle 類對象
析構了一個 Point 類對象
構造了一個 Point 類對象
複制構造了一個 Point 類對象
複制構造了一個 Point 類對象
複制構造了一個 Circle 類對象
複制構造了一個 Point 類對象
複制構造了一個 Rectangle 類對象
複制構造了一個 Point 類對象
複制構造了一個 Rectangle 類對象
複制構造了一個 Square 對象
f::面積是 0
f::面積是 3.14159
f::面積是 8
f::面積是 1
P[0]->面積是 0
P[1]->面積是 12.5664
P[2]->面積是 20
P[3]->面積是 9
析構了一個 Point 類對象
析構了一個 Circle 類對象
析構了一個 Point 類對象
析構了一個 Rectangle 類對象
析構了一個 Point 類對象
析構了一個 Square 對象
析構了一個 Rectangle 類對象
析構了一個 Point 類對象
析構了一個 Point 類對象
析構了一個 Circle 類對象
析構了一個 Point 類對象
析構了一個 Circle 類對象
析構了一個 Point 類對象
析構了一個 Rectangle 類對象
析構了一個 Point 類對象
析構了一個 Rectangle 類對象
析構了一個 Point 類對象
析構了一個 Square 對象
析構了一個 Rectangle 類對象
析構了一個 Point 類對象
析構了一個 Square 對象
析構了一個 Rectangle 類對象
析構了一個 Point 類對象
```

```
int main() {
	Square Squ1(0, 0, 1), Squ2(Point(2, 2), 3);
	Rectangle Rec1(2, 3, 2, 4), Rec2(Point(5, 6), 4, 5);
	Circle C1(5, 6, 1), C2(Point(1, 2), 2);
	Point P1(1, 2), * P[4] = { new Point(P1), new Circle(C2), new
Rectangle(Rec2), new Square(Squ2) };
	f(P1); f(C1); f(Rec1); f(Squ1);
	for (int i = 0; i < 4; i++) {
		cout << "P[" << i << "]->" << "面積是" << P[i]->GetArea() << endl;
	}
	for (int i = 0; i < 4; i++) {
		delete P[i];
	}
}
```

3.

C++ code

```
#include<iostream>
#include<cmath>
using namespace std;
#define pi acos(-1.0)
class Point {
private: double x, y;
public:
     Point(double xx = 0, double yy = 0) : x(xx), y(yy) { cout << "構造了一個Point類對象" << endl; }
     Point(const Point& P) : x(P.x), y(P.y) { cout << "複制構造了一個Point類對象" << endl; }
     virtual ~Point() { cout << "析構了一個Point類對象" << endl; }
     virtual double GetArea() { return 0.0; };
};
class Circle : virtual public Point {
private: double R;
public:
     Circle(double xx = 0, double yy = 0, double rr = 0) :Point(xx, yy), R(rr) { cout << "構造了一個Circle對象" << endl; }
     Circle(const Point& P, double rr = 0) :Point(P), R(rr) { cout << "構造了一個Circle對象" << endl; }
     Circle(const Circle& C) :Point(C), R(C.R) { cout << "複制構造了一個Circle對象" << endl; }
     ~Circle() { cout << "析構了一個Circle對象" << endl; }
     double GetArea() { return pi * R * R; }
};
class Square : virtual public Point {
private: double L;
public:
     Square(double xx = 0, double yy = 0, double ll = 0) :Point(xx, yy), L(ll) { cout << "構造了一個Square對象" << endl; }
     Square(const Point& P, double ll = 0) :Point(P), L(ll) { cout << "構造了一個Square對象" << endl; }
     Square(const Square& Squ) :Point(Squ), L(Squ.L) { cout << "複制構造了一個Square對象" << endl; }
     ~Square() { cout << "析構了一個Square對象" << endl; }
     double GetArea() { return L * L; }
};
class Margin : public Circle, public Square {
public:
     Margin(double xx = 0, double yy = 0, double rr = 0, double ll = 0) :Point(xx, yy), Circle(xx, yy, rr), Square(xx, yy, ll) {}
     Margin(const Point& P, double rr = 0, double ll = 0) :Point(P), Circle(P, rr), Square(P, ll) {}
     Margin(const Margin& M) :Point(M), Circle(M), Square(M) {}
     double GetArea() {
          if (2 * Circle::GetArea() >= pi * Square::GetArea()) { return Circle::GetArea() - Square::GetArea(); }
          else if (4 * Circle::GetArea() <= pi * Square::GetArea()) { return Square::GetArea() - Circle::GetArea(); }
          else return 0.0;
     }
};
int main() {
     Margin M1(0, 0, 3, 5), M2(1, 1, 6, 2), M3(3, 2, 1, 5);
     cout << "Margin[1] = " << M1.GetArea() << endl;
     cout << "Margin[2] = " << M2.GetArea() << endl;
     cout << "Margin[3] = " << M3.GetArea() << endl;
}
```

Result

```
構造了一個 Point 類對象
構造了一個 Circle 對象
構造了一個 Square 對象
構造了一個 Point 類對象
構造了一個 Circle 對象
構造了一個 Square 對象
構造了一個 Point 類對象
構造了一個 Circle 對象
構造了一個 Square 對象
Margin[1] = 0
Margin[2] = 109.097
Margin[3] = 21.8584
析構了一個 Square 對象
析構了一個 Circle 對象
析構了一個 Point 類對象
析構了一個 Square 對象
析構了一個 Circle 對象
析構了一個 Point 類對象
析構了一個 Square 對象
析構了一個 Circle 對象
析構了一個 Point 類對象
```